AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Functions and arguments

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

Before you start

A script sets capacity = 12 and used = 4. What does print(capacity - used) display?

Plan time to read, investigate and explain. Keep predictions separate from observations.

STUDENT LESSON · 1 OF 18

Your mission

> Start with the target: Distinguish a function definition from a function call. Try the recall before opening notes.

Your learning target

I can distinguish a **function definition** from a **function call**.

I can match positional **argument**s to **parameter**s in order.

I can reuse one **function** and test that it uses the supplied values.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Lists and indexing. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Branching and boundary tests. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Distinguish a **function definition** from a **function call**

Match positional **argument**s to **parameter**s in order

Reuse one **function** and test that it uses the supplied values

Retrieve what you already know

> A script sets capacity = 12 and used = 4. What does print(capacity - used) display?

My starting explanation

This week's end goal

Trace **function**s and **argument**s and explain the changed program before running it.

Notice the evidence

> Investigate the supplied Functions and arguments case. Separate observations, given facts and assumptions before choosing an explanation.

A fictional supply cupboard records capacity and the number of occupied spaces. The remaining-space calculation is capacity minus used.

Instead of copying the same instructions for every cupboard, define one **function** and **call** it with different values. A **function** makes a procedure reusable; its name alone does not guarantee that its instructions are correct.

Our **input** contract requires nonnegative integer counts with used no greater than capacity. The example assumes these conditions; it does not validate them. This week the **function** prints its result.

Week 9 will show how to return a value for later calculations.

> **List the given facts and one fact that is not supplied.**

See how it works

> Follow the mechanism for Functions and arguments. Connect each labelled input, deciding step and result.

```
def show_remaining(capacity, used):
    remaining = capacity - used
    print(remaining)
```

```
show_remaining(12, 4)
show_remaining(10, 7)
```

Arguments matched in order

CALL	PARAMETER VALUES	PRINTED OUTPUT
show_remaining(12, 4)	capacity 12; used 4	8
show_remaining(10, 7)	capacity 10; used 7	3

Each call runs the same body with the values supplied for that call.

def introduces the **definition**. The **function** name is followed by **parameter** names in parentheses and a colon. Four-space indentation marks the body. Executing the **definition** creates the **function** but does not run its body.

The two unindented **call**s then run it, producing 8 and 3 on separate lines. A blank line helps readability; indentation determines the block.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

STUDENT LESSON · 4 OF 18

Compare the cases

> Compare the supplied cases for Functions and arguments. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order. Explain what happens if the file contains only the **definition**.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change one thing

Change: Remove both **call**s and leave the **definition** only.

Predicted result

Prediction: No **output** is printed.

Why the result follows

Reason: The body runs only when the **function** is **call**ed.

STUDENT LESSON · 5 OF 18

Words that unlock the idea

Use the vocabulary of Functions and arguments to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
Function	A named block of reusable instructions that runs when called
Definition	Code that creates a function and specifies its body
Call	An instruction that runs a function with supplied arguments
Parameter	A name in a function definition that receives an argument
Argument	A value supplied when calling a function

Use two terms to explain a step in this week's example.

STUDENT LESSON · 6 OF 18

Follow a worked example

Follow the worked trace for Functions and arguments, predict a step, then compare with its explanation.

Execution event	What happens
Execute the definition	Create the function; no output yet
Call with 12 and 4	Match parameters; subtract; print 8
Call with 10 and 7	Match new values; subtract; print 3
After the second call	Continue with the next statement

Worked explanation · Functions and arguments

In show_remaining(12, 4), 12 and 4 are **argument**s; capacity and used are **parameter**s.

Positional **argument**s match from left to right. show_remaining(4, 12) produces -8, showing why order matters; it also violates our written **input** contract. Use a fresh file with the **definition** before its **call**s.

Predict the output of show_remaining(9, 9) followed by show_remaining(9, 0). Name the parameter receiving each argument. What would be printed if both calls were removed?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Functions and arguments

Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.

The facts we will use

Supplied facts

Python 3: def show_free(capacity,reserved): print(capacity-reserved). **Call**s: show_free(11,3); show_free(3,11).

AI ACADEMY / PYTHON-BRIDGE/8

Functions and arguments

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.

1. Define without calling

The definition creates the function but prints nothing yet.

2. Match first arguments

capacity=11 and reserved=3 give a printed 8.

3. Trace reversed call

capacity=3 and reserved=11 give -8 under the same arithmetic.

CHANGE ONE THING

Remove both calls and leave the definition only.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Define without calling	The definition creates the function but prints nothing yet.
2. Match first arguments	capacity=11 and reserved=3 give a printed 8.
3. Trace reversed call	capacity=3 and reserved=11 give -8 under the same arithmetic.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

Positional **argument**s match **parameter** names in order. A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

What this example does not establish

Keep the stated limit

The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

Keep the stated limit

Read the labelled picture: Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

Keep the stated limit

Change one thing: Remove both **call**s and leave the **definition** only. Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the Functions and arguments claim and explain a repair using the supplied evidence.

```
def show_remaining(capacity, used):
    remaining = 12 - 4
    print(remaining)

show_remaining(12, 4)
show_remaining(10, 7)
```

This faulty **function** prints 8 twice. Its fixed numbers ignore the **argument**s. Replace 12 - 4 with capacity - used, then rerun both **call**s.

Testing only the first **call** would miss the bug because its expected answer happens to be 8.

For this two-**parameter definition**, show_remaining(12) raises TypeError because an **argument** is missing. Correct the **call** to supply both values.

Naming a **function** without **call**ing it does not execute the body: the parentheses in show_remaining(12, 4) are part of the **call**.

Locate the earliest unsupported step

The faulty body calculates 12 - 4 regardless of the **call**. State the **output** and expected **output** for show_remaining(11, 2). Repair the expression and name an earlier **call** to rerun.

> **State one repair and the checks you would rerun.**

STUDENT LESSON · 8 OF 18

Practise together

> Practise Functions and arguments with support, then explain the deciding step in your own words.

In def show_remaining(capacity, used), identify the **function** name and **parameter**s. In show_remaining(15, 6), identify the **argument**s and their matching **parameter**s.

> **Predict the output of show_remaining(9, 9) followed by show_remaining(9, 0). Name the parameter receiving each argument. What would be printed if both calls were removed?**

Explain your prediction to a partner before comparing results.

Practise with less help: Follow the execution order

Use workbook W2 for the connected example "**Function**s and **argument**s". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for **Function**s and **argument**s. Proposed explanation: "Writing def runs the body immediately". Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

STUDENT LESSON · 9 OF 18

Run the investigation

> Investigate Functions and arguments. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order. Explain what happens if the file contains only the **definition**.

```
def show_remaining(capacity, used):
    remaining = capacity - used
    print(remaining)

show_remaining(12, 4)
show_remaining(10, 7)
```

```
def show_remaining(capacity, used):
    remaining = 12 - 4
    print(remaining)

show_remaining(12, 4)
show_remaining(10, 7)
```

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in Functions and arguments. Explain why each step is justified.

Explain → apply → challenge

In def show_remaining(capacity, used), identify the **function** name and **parameter**s. In show_remaining(15, 6), identify the **argument**s and their matching **parameter**s.

The faulty body calculates 12 - 4 regardless of the **call**. State the **output** and expected **output** for show_remaining(11, 2). Repair the expression and name an earlier **call** to rerun.

The correct subtraction **function** prints -1 for show_remaining(4, 5). Does successful execution make this a valid cupboard record? Explain what the **function** assumes and what needs checking.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Python readiness portfolio. Keep an annotated decision and the evidence supporting it.

Add a labelled **function**s and **argument**s **evidence** sheet to Python readiness portfolio. Show how you can distinguish a **function definition** from a **function call**.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

For each **call**, record the supplied **argument**s, expected printed **output** and observed **output**. Include different values that produce different answers, plus a valid zero-result case.

Call order controls **output** order; it does not change the subtraction rule. These **function**s use supplied data and written instructions, not a pattern learned from examples.

> **A learner tests only show_remaining(12, 4) and says the function works for every cupboard. Give a second valid call that exposes the fixed-number bug, state its expected output and explain what passing two tests still cannot prove.**

The correct subtraction **function** prints -1 for show_remaining(4, 5). Does successful execution make this a valid cupboard record? Explain what the **function** assumes and what needs checking.

My limited claim and a useful next check

STUDENT LESSON · 13 OF 18

Study a complete case

> Revisit the complete Functions and arguments case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

Arguments matched in order

CALL	PARAMETER VALUES	PRINTED OUTPUT
show_remaining(12, 4)	capacity 12; used 4	8
show_remaining(10, 7)	capacity 10; used 7	3

Each call runs the same body with the values supplied for that call.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

STUDENT LESSON · 14 OF 18

Try a new case independently

> Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Match positional arguments to parameters

Predict both returned and printed values, showing which **argument** supplies first and second. What is displayed if the program contains only the **function definition** and no **call**s?

Why does swapping these **argument**s change the result?

```
def difference(first, second):
    return first - second
print(difference(11, 4))
print(difference(4, 11))
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Each sealed box contains 6 counters. Define show_stock(boxes, loose) to print the total counters using its two **parameter**s. Assume nonnegative integer **argument**s. **Call** it with (3, 2), (0, 2) and (3, 0).

Predict each **output** and explain why show_stock(2, 3) is a different case.

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

Your lesson route

> Use this map to navigate Functions and arguments, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.

By the end, I can	How I will show it
Distinguish a function definition from a function call	Explain the deciding step, show a trace or test, and state one limit.
Match positional arguments to parameters in order	Explain the deciding step, show a trace or test, and state one limit.
Reuse one function and test that it uses the supplied values	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain Functions and arguments without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Distinguish a **function definition** from a **function call**

> **My deciding step, evidence and one remaining question**

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? capacity=3 and reserved=11 give -8 under the same arithmetic.

1. Match first **argument**s

2. Trace reversed **call**

3. Define without **call**ing

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Trace reversed **call** capacity=3 and reserved=11 give -8 under the same arithmetic. Positional **argument**s match **parameter** names in order.

A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Remove both **call**s and leave the **definition** only. Which conclusion is supported by the supplied facts?

1. This one example guarantees correct results for every future **input**.

2. We can decide the result without examining the changed **input** or the procedure.

3. No **output** is printed.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: No **output** is printed. The body runs only when the **function** is **call**ed.

Repair a missing building block

Review Lists and indexing (python-bridge/7). Goal: Distinguish a list index from the value stored at that position. Short practice: Label the three index positions.

Predict all original **output**s, then name the first **output** and stopping point after using index 3.. Return to this week and explain what you changed.

Review Branching and boundary tests (python-bridge/5). Goal: Trace the selected branch in an if elif else chain. Short practice: Trace -2, 4 and 5 through the branches.

Explain why the negative check comes first and what label -2 gets if it is removed.. Return to this week and explain what you changed.

Review Values variables and types (python-bridge/2). Goal: Distinguish integer, floating-point, text and Boolean values. Short practice: Predict both print **output**s and explain why they differ.

Does count+tag work, and what happens to each valid **output** after count becomes ten?. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

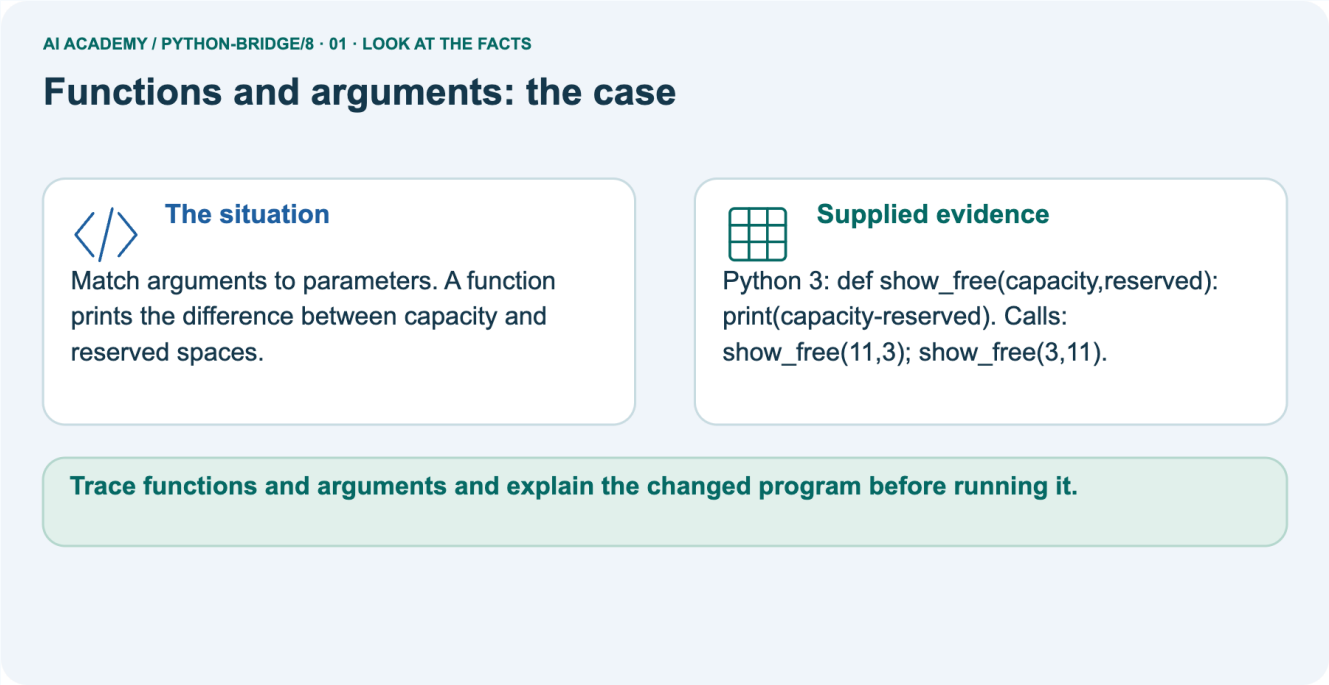
Drafts stay in this browser when storage is available; clear drafts on a shared device.

STUDENT LESSON · 18 OF 18

Visual concept journey

> Explain each Functions and arguments graphic using its labels, deciding evidence and relevant limitation.

01. Functions and arguments: the case



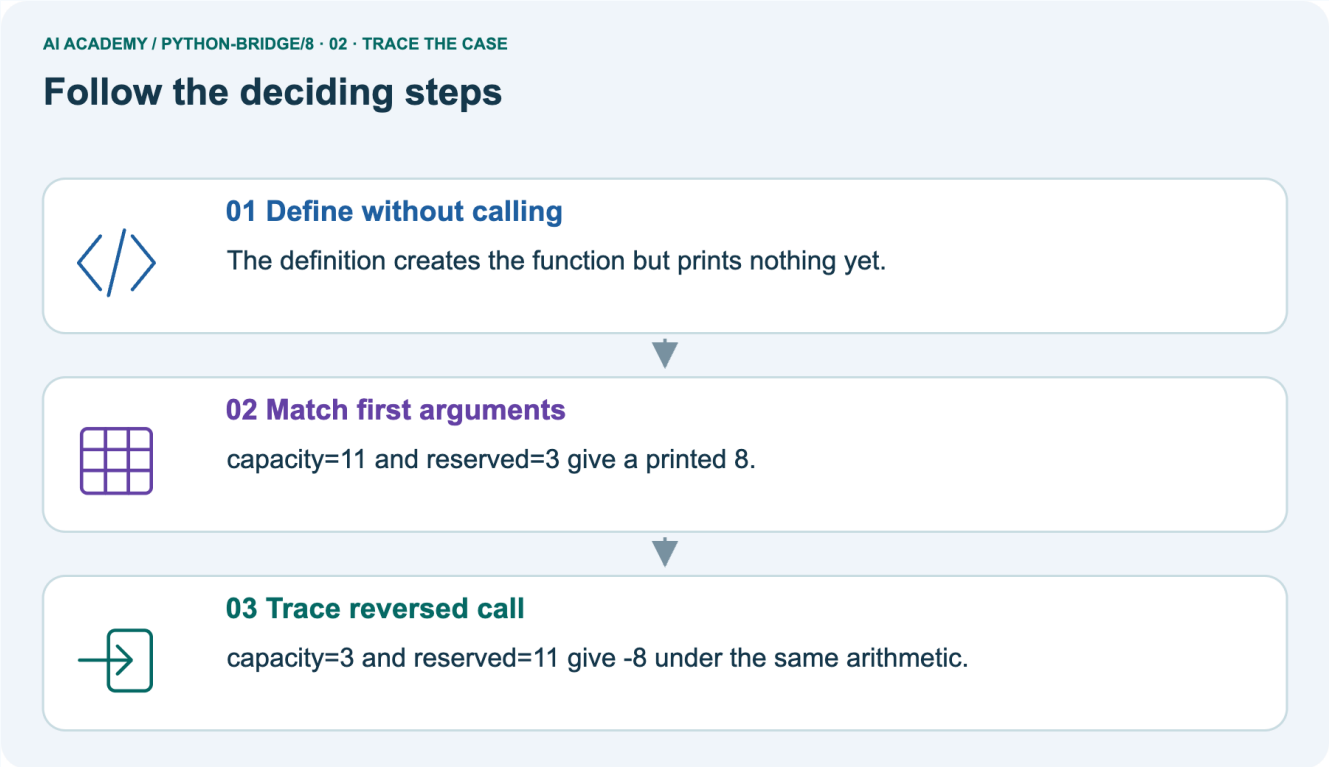
Follow the labelled picture with the matching explanation. Open the vector diagram

Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces. Supplied facts: Python 3: def show_free(capacity,reserved): print(capacity-reserved).

Calls: show_free(11,3); show_free(3,11).

> Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Define without **call**ing: The **definition** creates the **function** but prints nothing yet. → Match first **argument**s: capacity=11 and reserved=3 give a printed 8. → Trace reversed **call**: capacity=3 and reserved=11 give -8 under the same arithmetic.

> Point to each numbered stage and explain which recorded **input** it uses.

03. Arguments matched in order

AI ACADEMY / PYTHON-BRIDGE/8 · 03 · INSPECT THE LESSON MODEL

Arguments matched in order

Call	Parameter values	Printed output
show_remaining(12, 4)	capacity 12; used 4	8
show_remaining(10, 7)	capacity 10; used 7	3

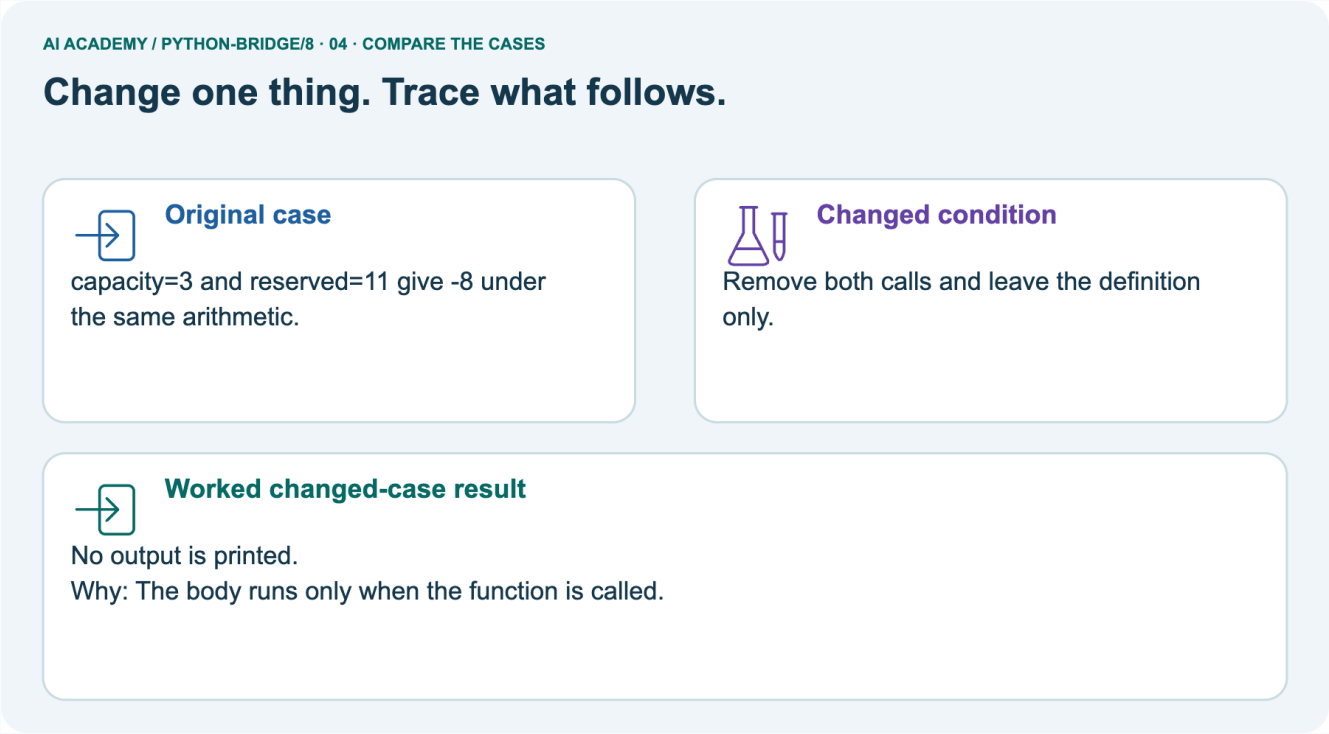
Each call runs the same body with the values supplied for that call.

Follow the labelled picture with the matching explanation. Open the vector diagram

Arguments matched in order. show_remaining(12, 4) | capacity 12; used 4 | 8. show_remaining(10, 7) | capacity 10; used 7 | 3

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



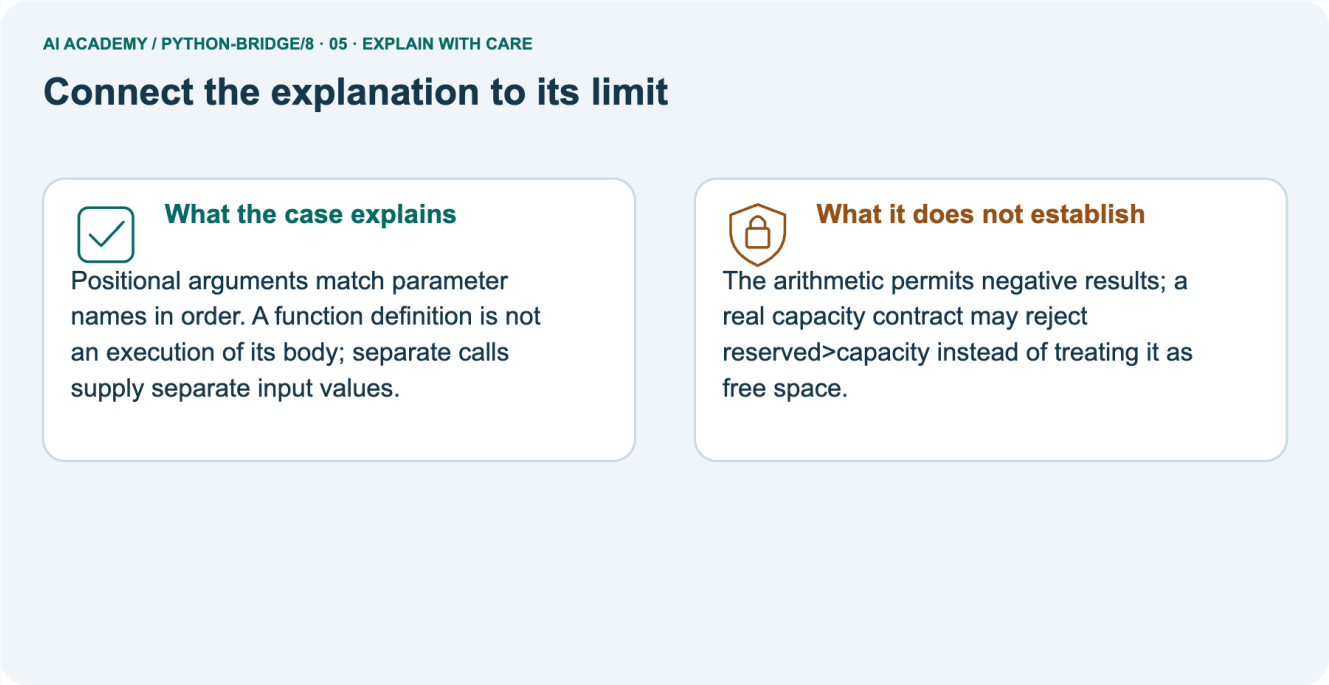
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: capacity=3 and reserved=11 give -8 under the same arithmetic. Changed condition: Remove both **call**s and leave the **definition** only. Expected result: No **output** is printed.

Reason: The body runs only when the **function** is **call**ed.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



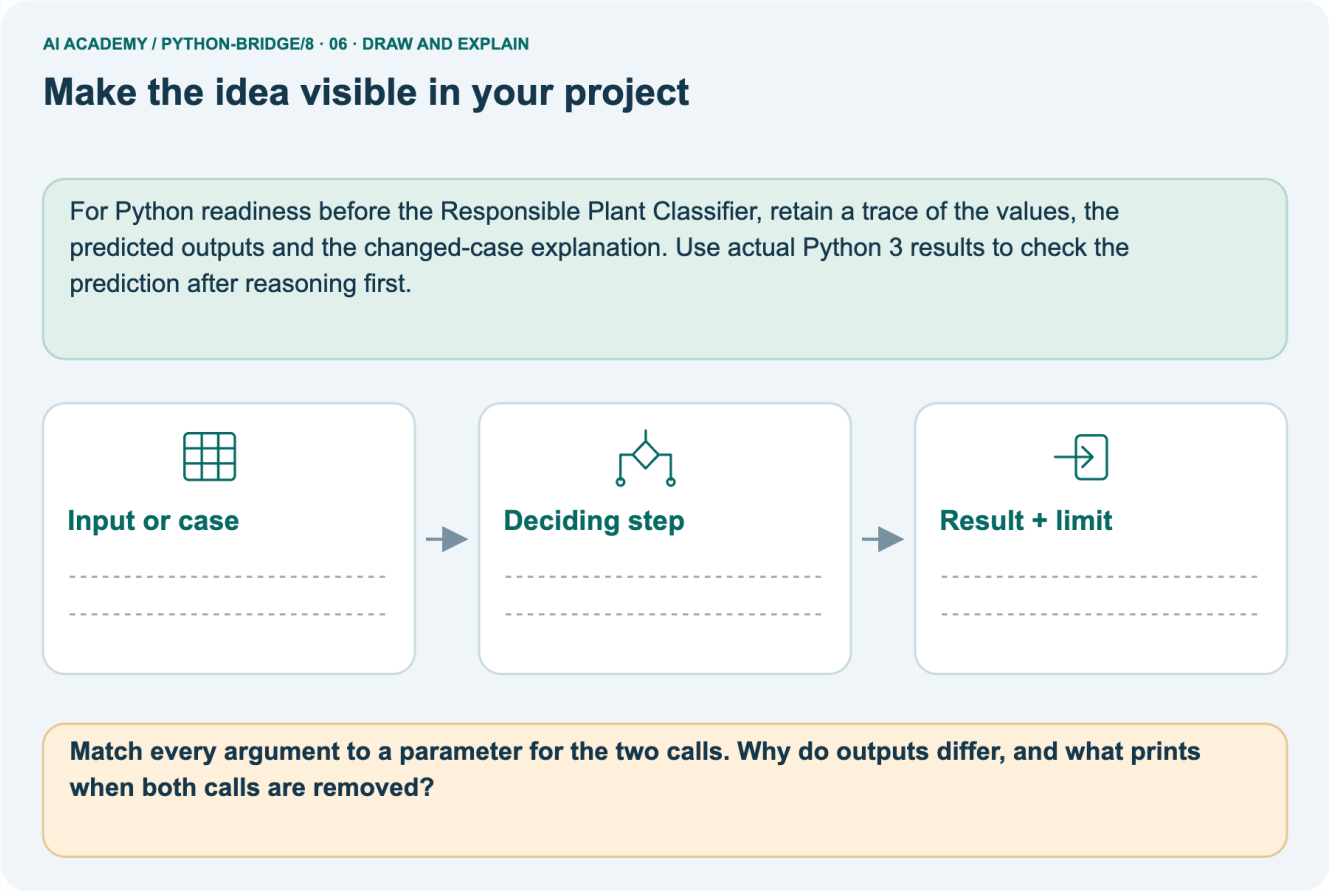
Follow the labelled picture with the matching explanation. Open the vector diagram

Positional **argument**s match **parameter** names in order. A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

Limit: The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.